EXPERIMENT 8

Implementation of Apache Spark

Step-by-Step Practice Guide for Ubuntu

Aim	To implement Apache Spark in Ubuntu.
What you will practice	Update Ubuntu, verify Java, install Scala, set Spark PATH, start spark-shell, load an input file, run WordCount, save output, and view the result.
Main commands	apt, nano, source, spark-shell, Scala RDD operations, cat

Before you start: Your record assumes Spark is available at `/usr/local/spark`. It does not show the Spark download/extraction step. If your lab VM already contains that folder, continue directly. If it does not, your instructor/lab setup must provide or install Spark before the PATH step.

Version warning from the record: the screenshots show different Java/Scala versions in different places (for example JDK 8 installation, a Java 11 version display, Scala 2.11, and Spark shell using Scala 2.12). Do not try to force all screenshot versions to match. For practice, use the versions already configured in your lab VM and confirm that `spark-shell` starts successfully.

Step-by-Step Procedure

Step 1: Update Ubuntu packages

Open Terminal and update the package list.

```
sudo apt update
```

Wait until the update completes without fatal errors.

Step 2: Check / install Java

The first screenshot in the record shows installation of OpenJDK 8 and then checks the Java version.

```
sudo apt install openjdk-8-jdk
java -version
```

If Java is already installed in your lab VM, you only need to verify it with `java -version`.

Step 3: Install Scala

Install Scala and verify that the command is available.

```
sudo apt install scala
scala -version
```

Step 4: Set Spark environment variables in .bashrc

Open your shell configuration file.

```
nano ~/.bashrc
```

Add these two lines at the end, exactly as shown in the record:

```
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

Save and exit Nano: **Ctrl+O**, press **Enter**, then **Ctrl+X**. Apply the changes:

```
source ~/.bashrc
```

Optional verification:

```
echo $SPARK_HOME
which spark-shell
```

Step 5: Start the Spark shell

Launch the interactive Spark/Scala shell.

```
spark-shell
```

A successful start should display the Spark banner and eventually give you a `scala>` prompt. The record screenshot shows Spark 3.5.6, but your installed version may differ.

Step 6: Import the Spark classes shown in the record

At the `scala>` prompt, enter:

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark._
```

In many Spark shells these imports are not strictly required for a simple word count, but they are part of your experiment record, so practice them.

Step 7: Get the SparkContext

The Spark shell already creates a SparkSession named `spark`. The record obtains its SparkContext like this:

```
val sc = spark.sparkContext
```

You should see a SparkContext object returned.

Step 8: Prepare the input text file

This helper is added for practice; the record only shows loading input.txt and does not show how the file was created.

If you do not already have `input.txt`, open a second terminal in the directory where you started `spark-shell` and create a small file:

```
echo 'hello this is aswin world hello this is aswin world' > input.txt
```

This sample text is chosen only so the output resembles the word-count screenshot in the record.

Step 9: Load the input file into an RDD

Back in the Spark shell:

```
val input = sc.textFile("input.txt")
```

Spark creates an RDD representing the lines of the file. If Spark says the file does not exist, check the working directory and file name.

Step 10: Apply the WordCount logic

Enter the WordCount transformation shown in the record:

```
val count = input.flatMap(line => line.split(" ")).map(word => (word, 1)).reduceByKey(_ + _)
```

Meaning: split each line into words -> convert each word to `(word,1)` -> add counts for identical words.

Step 11: Save results to an output directory

Save the RDD as text files:

```
count.saveAsTextFile("output")
```

Important: Spark expects the target folder not to already exist. If `output` exists from an earlier run, remove or rename it before running this command again.

Step 12: Exit the Spark shell and view the output

Leave the Spark shell. The record screenshot shows `exit`; depending on your shell, `:quit` also exits.

```
:quit
```

Then, in the normal Ubuntu terminal, view the generated part file:

```
cat output/part-00000
```

The record shows word/count pairs such as `(is,2)`, `(hello,2)`, `(aswin,1)`, `(this,2)`, and `(world,2)`. Your exact order can differ because distributed output ordering is not guaranteed.

Quick Rehearsal - Exam/Lab Sequence

#	Stage	What to type / remember
1	Update	`sudo apt update`
2	Java	`java -version`
3	Scala	`scala -version`
4	PATH	`nano ~/.bashrc -> add SPARK_HOME and PATH -> source ~/.bashrc`
5	Start	`spark-shell`
6	Context	`val sc = spark.sparkContext`
7	Load	`val input = sc.textFile("input.txt")`
8	WordCount	`val count = input.flatMap(line => line.split(" ")).map(word => (word,1)).reduceByKey(_ + _)`
9	Save	`count.saveAsTextFile("output")`
10	View	`cat output/part-00000`

What each WordCount operation does

Operation	Purpose
`textFile("input.txt")`	Reads the file into an RDD of lines.
`flatMap(line => line.split(" "))`	Splits every line into individual words and flattens them.
`map(word => (word, 1))`	Creates one key-value pair for every word.
`reduceByKey(_ + _)`	Adds the 1s for identical words to produce final counts.
`saveAsTextFile("output")`	Writes the result as one or more part files inside an output directory.

Common Problems and Fixes

spark-shell: command not found
Check that /usr/local/spark exists, verify SPARK_HOME in ~/.bashrc, run source ~/.bashrc, then run which spark-shell.

input.txt not found
The file path is resolved from the directory where spark-shell is running. Create the file there or use the correct path.

FileAlreadyExistsException for output
Delete the old output directory before repeating the job: rm -rf output. Then run saveAsTextFile again.

Java/Scala version looks different from the record
The record itself shows mixed versions. The key practice check is that spark-shell starts and the word-count commands execute.

part-00000 missing
List the output directory with ls output. Spark may produce a different part filename depending on partitions/version.

Expected Result

Apache Spark is started successfully, the input text is loaded into an RDD, the WordCount transformation is executed, the result is saved to an output directory, and the generated word counts are displayed. This matches the experiment result: implementation of Apache Spark was completed successfully.

Viva - Very Short Answers

Q: What is Apache Spark?
A: A distributed data-processing framework designed for fast large-scale computation.

Q: What is an RDD?
A: Resilient Distributed Dataset - Spark's fault-tolerant distributed collection abstraction.

Q: What does flatMap do here?
A: It splits lines into words and returns one flattened collection of words.

Q: Why reduceByKey?
A: To add all values belonging to the same word/key.

Q: What is SparkContext?
A: The object that represents the connection to Spark and is used to create/access RDDs.